Aggregate Documentation Report for `utils_jin`

STAT 221 Bayesian Poker Project

May 1, 2026

Contents

1 Purpose and Scope **3**

2 High-Level Architecture **3**
- 2.1 Main Data Flow . . . 3
- 2.2 Core Mathematical Objects . . . 3
- 2.3 Bayesian Filtering Equation . . . 4
- 2.4 EM Preparation . . . 4

3 Package Inventory **4**

4 `__init__.py` **5**
- 4.1 Role . . . 5
- 4.2 Why It Exists . . . 5

5 `compat.py` **5**
- 5.1 Problem Solved . . . 5
- 5.2 Functions . . . 5
- 5.3 Design Note . . . 6

6 `cards.py` **6**
- 6.1 Role . . . 6
- 6.2 Constants . . . 6
- 6.3 Important Functions . . . 6
- 6.4 Examples . . . 7
- 6.5 Model Connection . . . 7

7 `action_encoder.py` **7**
- 7.1 Role . . . 7
- 7.2 Action Buckets . . . 7
- 7.3 Sizing Rule . . . 8
- 7.4 Key Data Class . . . 8
- 7.5 Important Functions . . . 8

8 state_encoder.py **9**
8.1 Role . . . 9
8.2 StateKey . . . 9
8.3 Important Functions . . . 9
8.4 Design Tradeoff . . . 10

9 records.py **10**
9.1 Role . . . 10
9.2 DecisionRecord . . . 10
9.3 Important Functions . . . 10
9.4 Why This Is Necessary . . . 11

10 gto_prior.py **11**
10.1 Role . . . 11
10.2 Action Probability Interface . . . 11
10.3 Hand Features . . . 11
10.4 Heuristic Logic . . . 12
10.5 Theta Connection . . . 12
10.6 EM Prior Counts . . . 12

11 preflop_filter.py **12**
11.1 Role . . . 12
11.2 Classes . . . 13
11.3 Update . . . 13
11.4 Effective Sample Size . . . 13
11.5 Convenience Functions . . . 13

12 em_preflop.py **13**
12.1 Role . . . 13
12.2 E-Step . . . 14
12.3 Soft Counts . . . 14
12.4 M-Step Helper . . . 14
12.5 Evaluation Helper . . . 14
12.6 What Is Still Missing for Full EM . . . 14

13 README.md **15**
13.1 Role . . . 15
13.2 Minimal Filtering Example . . . 15
13.3 Minimal EM Starter Example . . . 15

14 Current Smoke-Test Results **15**

15 How the Scripts Fit Together **16**
15.1 Filtering Workflow . . . 16
15.2 EM Starter Workflow . . . 16

16 Recommended Next Engineering Steps **17**

17 Summary **17**

1 Purpose and Scope

The `utils_jin/` package is the first complete implementation layer for running pre-flop Bayesian range inference and preparing the project for EM. It does not replace the original `utils/` folder. Instead, it wraps and extends the existing parser so that the model code has clean, compact objects:

$$\begin{aligned}\text{PHH sessions} &\to \text{parsed hands} \to \text{decision records}\\ &\to \text{state/action encodings} \to \text{pre-flop filtering}\\ &\to \text{EM soft counts.}\end{aligned}$$

The package currently focuses on pre-flop inference. It includes post-flop state keys and records because the data extractor already sees all streets, but the implemented filter and EM starter are pre-flop only. This is intentional: pre-flop filtering is the first working model, and post-flop strength filtering should plug into the same record/state infrastructure later.

2 High-Level Architecture

2.1 Main Data Flow

The current intended data flow is:

1. `compat.py` makes the legacy `utils/parse.py` parser importable.
2. `records.py` parses hands and produces one `DecisionRecord` per player action.
3. `action_encoder.py` recomputes model-facing action buckets.
4. `state_encoder.py` maps rich parser states to small discrete `StateKey` values.
5. `cards.py` provides 1326-combo and 169-class card utilities.
6. `gto_prior.py` provides a simple heuristic $p_{\mathrm{GTO}}(a \mid h, s)$.
7. `preflop_filter.py` updates a range $R_t(h)$ using Bayesian filtering.
8. `em_preflop.py` turns filtering posteriors into EM responsibilities and soft counts.

2.2 Core Mathematical Objects

Object	**Meaning**	**Implemented In**
$h \in \mathcal{H}$	169 pre-flop hand class	`cards.py`
$s \in \mathcal{S}$	compact public state key	`state_encoder.py`
$a \in \mathcal{A}$	action bucket	`action_encoder.py`
$p_{\mathrm{GTO}}(a \mid h, s)$	heuristic pre-flop action prior	`gto_prior.py`
$R_t(h)$	filtered range after action t	`preflop_filter.py`
$q_m(h)$	EM responsibility for player-hand m	`em_preflop.py`

2.3 Bayesian Filtering Equation

The pre-flop filter performs the recursive update:

$$R_t(h) = \frac{R_{t-1}(h)\,\phi_{h,s_t}[a_t]}{\sum_{h' \in \mathcal{H}} R_{t-1}(h')\,\phi_{h',s_t}[a_t]}.$$

In the current implementation,

$$\phi_{h,s}[a] = p_{\mathrm{GTO}}(a \mid h, s),$$

where p_{GTO} is a transparent heuristic, not a solver output. The interface is designed so that a real solver table can later replace this heuristic without rewriting the filter.

2.4 EM Preparation

The implemented EM starter uses the filter as the E-step engine. For each player-hand group m, it computes:

$$q_m(h) = \mathbb{P}(V_m = h \mid a^m_{1:T_m}, s^m_{1:T_m}).$$

Then it forms soft counts:

$$N_{h,s,a} = \sum_m \sum_{t \in \mathrm{pre}} q_m(h)\,\mathbf{1}\{s^m_t = s,\ a^m_t = a\}.$$

Finally, the M-step helper computes MAP-style action likelihoods:

$$\phi_{h,s,a} \propto N_{h,s,a} + \lambda p_{\mathrm{GTO}}(a \mid h, s) + \epsilon.$$

3 Package Inventory

File	Lines	Primary Role
`__init__.py`	13	Package description and namespace marker.
`compat.py`	48	Compatibility wrapper for importing legacy parser modules.
`cards.py`	207	Card normalization, 1326 combos, 169 classes, priors.
`action_encoder.py`	153	Model-facing action buckets using pre-action context.
`state_encoder.py`	203	Compact public state keys for filtering and EM.
`records.py`	224	Flat decision-record extraction from parsed hands.
`gto_prior.py`	218	Simple GTO-inspired pre-flop action prior.
`preflop_filter.py`	124	Bayesian range filter over 169 classes.
`em_preflop.py`	139	E-step, soft-count, and M-step helpers.
`README.md`	47	Short usage guide.

4 __init__.py

4.1 Role

__init__.py turns utils_jin/ into a Python package and documents the purpose of the package. It intentionally contains no runtime logic. Its docstring states that the package provides:

- card and combo helpers;
- compact public state encoders;
- a simple GTO-inspired pre-flop action prior;
- a pre-flop Bayesian range filter;
- EM starter functions.

4.2 Why It Exists

Keeping utils_jin/ as a package means downstream code can import:

```
from utils_jin.records import extract_session_records
from utils_jin.preflop_filter import filter_preflop_records
```

This is cleaner than relying on notebook-relative imports.

5 compat.py

5.1 Problem Solved

The existing parser in utils/parse.py imports hand_map and action_map as top-level modules:

```
from hand_map import poker_hand_mapper
from action_map import classify
```

That works when a notebook runs from inside utils/, but it is fragile when importing from the repository root. compat.py solves this without editing legacy code by temporarily adding the old utils/ directory to sys.path.

5.2 Functions

repo_root()
Returns the repository root based on the location of utils_jin/compat.py.

ensure_legacy_utils_path()
Adds the legacy utils/ path to sys.path if needed.

load_legacy_session_class()
Imports and returns the original Session class from utils/parse.py.

load_legacy_hand_mapper()
Imports and returns the original poker_hand_mapper.

5.3 Design Note

This file is a reconciliation layer. It avoids touching the old parser while making the new package robust. If the legacy parser is later converted into a proper package, this compatibility module can be simplified or removed.

6 `cards.py`

6.1 Role

`cards.py` defines the card universe and the mapping between exact two-card combinations and 169 pre-flop classes. It supports both the modeling view and the poker view:

- exact combo space: $\binom{52}{2} = 1326$;
- equivalence-class space: 169 classes such as `AA`, `AKs`, and `AKo`.

The current pre-flop filter tracks 169 classes. Exact combos are still needed for future post-flop blocker-aware mapping.

6.2 Constants

Name	Meaning
`RANKS_LOW_TO_HIGH`	Rank order from 2 through A.
`RANKS_HIGH_TO_LOW`	Rank order from A through 2.
`SUITS`	Suit order `s`, `h`, `d`, `c`.
`RANK_TO_VALUE`	Rank-to-integer map, A = 14.
`VALUE_TO_RANK`	Inverse map.

6.3 Important Functions

`normalize_card(card)`
Standardizes cards to rank-uppercase and suit-lowercase, e.g. `AS` becomes `As`.

`split_cards(cards)`
Converts compact strings such as `AsKd` into `["As", "Kd"]`.

`full_deck()`
Returns all 52 cards in stable order.

`all_combos()`
Returns all 1326 exact two-card combos.

`combo_to_class(combo)`
Maps exact cards to a 169 class.

`all_169_classes()`
Returns all 169 pre-flop classes.

`class_multiplicity(hand_class)`
Returns 6 for pairs, 4 for suited non-pairs, and 12 for offsuit non-pairs.

`initial_class_prior()`
Returns the class prior $C(h)/1326$.

`legal_combos(dead_cards)`
Removes exact combos blocked by board or known cards.

`aggregate_combo_probs(combo_probs)`
Aggregates exact-combo probabilities into 169-class probabilities.

6.4 Examples

```
from utils_jin.cards import combo_to_class, all_combos, initial_class_prior

combo_to_class("AsKs") # "AKs"
combo_to_class("AsKd") # "AKo"
combo_to_class("AhAd") # "AA"

len(all_combos()) # 1326
sum(initial_class_prior().values()) # 1.0
```

6.5 Model Connection

The initial range R_0 is:

$$R_0(h) = \frac{C(h)}{1326}, \qquad C(h) = \begin{cases} 6 & \text{pair,} \\ 4 & \text{suited non-pair,} \\ 12 & \text{offsuit non-pair.} \end{cases}$$

This is implemented by `initial_class_prior()`.

7 `action_encoder.py`

7.1 Role

`action_encoder.py` converts legacy parser action tuples into clean model-facing action buckets. The legacy parser already stores an action bucket, but it computes bet/raise size using the pot after the action. This script recomputes the bucket from the pre-action state, which is more appropriate for modeling.

7.2 Action Buckets

ID	Constant	Label
0	`FOLD`	`fold`
1	`CHECK_CALL`	`check_call`
2	`SMALL_BET_RAISE`	`small_bet_raise`
3	`MEDIUM_BET_RAISE`	`medium_bet_raise`
4	`LARGE_BET_RAISE`	`large_bet_raise`

7.3 Sizing Rule

The code uses different sizing conventions pre-flop and post-flop:

- **Pre-flop**: bucket by total raise size in big blinds. This avoids classifying a normal 2.25bb open as "large" merely because the blind pot is only 1.5bb.
- **Post-flop**: bucket by contribution divided by pot before action.

  The current pre-flop thresholds are:

$$\begin{cases} \text{small} & \text{if raise size} \leq 3\text{bb}, \\ \text{medium} & \text{if } 3\text{bb} < \text{raise size} \leq 8\text{bb}, \\ \text{large} & \text{if raise size} > 8\text{bb}. \end{cases}$$

  The current post-flop thresholds are:

$$\text{pot fraction} = \frac{\text{new chips contributed}}{\text{pot before action}},$$

$$\begin{cases} \text{small} & \text{if fraction} < 0.5, \\ \text{medium} & \text{if } 0.5 \leq \text{fraction} < 1.0, \\ \text{large} & \text{if fraction} \geq 1.0. \end{cases}$$

7.4 Key Data Class

`ActionEncoding` stores:

- `bucket`: model action bucket id;
- `label`: readable action label;
- `amount_to`: total amount committed to on the street;
- `contribution`: new chips added by the actor;
- `pot_fraction`: contribution divided by pot before action;
- `legacy_bucket`: original parser bucket.

7.5 Important Functions

`committed_before_action(state, hand, street, player)`
Reconstructs how much the player has already committed this street. It handles pre-flop blinds explicitly because blinds are not stored in the betting history.

`encode_action(state, hand, street, legacy_action)`
Produces an `ActionEncoding` from a pre-action state and legacy action.

8 state_encoder.py

8.1 Role

state_encoder.py compresses the rich parser state into a small discrete StateKey. This is essential. EM cannot work if every exact betting history is treated as a different state. The state encoder deliberately uses coarse buckets.

8.2 StateKey

StateKey contains seven fields:

Field	Meaning
street	pre-flop, flop, turn, or river
position	coarse position bucket
active_players	heads-up, three-way, or multiway
facing_bet	whether the player must call a bet
raise_count	0, 1, or 2+ raises this street
spr	stack-to-pot ratio bucket
board_texture	no board, dry, paired, monotone, connected, etc.

The string form is:

```
street|position|active_players|facing_bet|raise_count|spr|board_texture
```

8.3 Important Functions

position_bucket(player, num_players)
Uses the parser's seat assumptions: p1 is small blind and p2 is big blind.

active_count_bucket(count)
Buckets active player count into heads_up, three_way, or multiway.

raise_count_bucket(state)
Reads the parser's street action level and buckets into raises_0, raises_1, or raises_2plus.

stack_to_pot_bucket(state, player)
Buckets SPR into spr_low, spr_mid, or spr_high.

board_texture_bucket(board)
Produces a coarse board texture label.

current_bet_to_call(state, hand, street)
Reconstructs the current amount to call before the action.

facing_bet_bucket(state, hand, street, player)
Compares the current bet to the player's committed amount.

encode_state(state, hand, street, player, action_encoding)
Returns the final StateKey.

8.4 Design Tradeoff

The current state key is intentionally simple. It should be good enough to begin filtering and EM. If the state space is too coarse, the model will miss useful strategic context. If it is too detailed, likelihood tables will be sparse and EM will become unstable.

9 `records.py`

9.1 Role

`records.py` is the bridge between parsing and modeling. It converts nested legacy parser objects into a flat list of `DecisionRecord` rows. Every row corresponds to one player decision.

9.2 `DecisionRecord`

The `DecisionRecord` data class stores:

Field	Meaning
`session_id`	Source session directory.
`hand_id`	Hand id from PHH metadata when available.
`hand_index`	Index within parsed session.
`player_id`	Acting player, e.g. `p4`.
`street`	Street of the decision.
`decision_index`	Index within that street.
`board`	Public board before the action.
`pot_before`	Pot before the action.
`stack_before`	Acting player's stack before the action.
`active_player_count`	Number of players still active.
`state_key`	Structured `StateKey`.
`state_key_str`	String form of `StateKey`.
`action_bucket`	Model action bucket.
`action_label`	Readable action label.
`amount_to`	Total amount committed to on current street.
`contribution`	New chips added by the action.
`pot_fraction`	Contribution divided by pot before action.
`legacy_action_bucket`	Original parser action bucket.
`hole_cards`	True hole cards when known.
`hand_class`	True 169 class when known.
`strength_bucket`	True post-flop strength bucket when board exists.

9.3 Important Functions

`iter_hand_decision_records(hand, session_id, hand_index)`
Yields records from one parsed hand.

`extract_session_records(session_path)`
Parses one session directory and returns all decision records.

`extract_many_session_records(root_path)`
Extracts records from every session under a root directory.

`group_records_by_player_hand(records)`
Groups rows by (session, hand, player).

`preflop_only(records)`
Filters records to pre-flop actions.

9.4 Why This Is Necessary

The parser stores actions and states separately:

```
hand.states[street][t]
hand.actions[street][t]
```

The model needs them joined. `records.py` performs that join and adds the model encodings. This is the most important infrastructure layer for starting EM.

10 gto_prior.py

10.1 Role

`gto_prior.py` implements a simple, transparent approximation to

$$p_{\mathrm{GTO}}(a \mid h, s).$$

It is not a real solver output. Its purpose is to provide a reasonable pre-flop action likelihood interface so filtering can start immediately. Later, the same interface can be backed by actual solver frequencies or a better pre-flop chart.

10.2 Action Probability Interface

The main class is `PreflopGTOPrior`. Its most important method is:

```
action_probs(hand_class, state_key) -> dict[int, float]
```

It returns probabilities over all five action buckets.

10.3 Hand Features

`HandClassFeatures` stores:

- high rank;
- low rank;
- pair indicator;
- suited indicator;
- rank gap;
- number of broadway cards;

- ace indicator;
- scalar heuristic strength.

The helper `hand_class_features(hand_class)` computes these features.

10.4 Heuristic Logic

The prior uses:

- intrinsic hand strength;
- position adjustment;
- multiway penalty;
- prior raise-count penalty;
- premium-hand boost;
- speculative suited/connected boost.

It then builds action scores and applies a softmax. A small probability floor is mixed into every action. That floor is important: if an action has probability zero, Bayesian filtering would permanently eliminate some hands.

10.5 Theta Connection

This module does not yet implement a learned θ. However, it is the natural place where the current model's GTO perturbation will enter. The future version can use:

$$\phi_{h,s,\theta}[a] \propto p_{\mathrm{GTO}}(a \mid h, s) \exp\{u_\theta(h, s, a)\}.$$

Currently, `aggression` and `tightness` are simple manual knobs, not learned latent variables.

10.6 EM Prior Counts

`dirichlet_alpha(hand_class, state_key, kappa)` returns:

$$\alpha_{h,s,a} = \kappa p_{\mathrm{GTO}}(a \mid h, s).$$

These pseudo-counts are used by `em_preflop.py` in the M-step helper.

11 `preflop_filter.py`

11.1 Role

`preflop_filter.py` implements Bayesian range filtering over 169 pre-flop classes.

11.2 Classes

`FilterStep`
Debug record for one update. It stores the state key, observed action, evidence, ESS, top class, and top probability.

`PreflopRangeFilter`
Maintains and updates the current range $R_t(h)$.

11.3 Update

`PreflopRangeFilter.update(state_key, action_bucket)` performs:

$$\widetilde{R}_t(h) = R_{t-1}(h)\phi_{h,s_t}[a_t],$$

$$R_t(h) = \frac{\widetilde{R}_t(h)}{\sum_{h'} \widetilde{R}_t(h')}.$$

The denominator is stored as `evidence`; it is the predictive probability of the observed action under the current range and prior model.

11.4 Effective Sample Size

`effective_sample_size(distribution)` computes:

$$\mathrm{ESS}(R) = \frac{1}{\sum_h R(h)^2}.$$

ESS is a concentration diagnostic. A uniform range over 169 classes has ESS 169. A highly concentrated range has lower ESS.

11.5 Convenience Functions

`filter_preflop_records(records)`
Runs the filter over one grouped player-hand record sequence.

`uniform_class_distribution()`
Returns a uniform distribution over 169 classes for sensitivity checks.

12 `em_preflop.py`

12.1 Role

`em_preflop.py` provides the starter pieces needed for pre-flop EM. It does not run a full multi-iteration training loop yet. Instead, it exposes the core mathematical operations separately so they can be inspected and tested.

12.2 E-Step

`e_step_preflop(records, prior_model)` groups records by (session, hand, player), runs the preflop filter on each group, and returns:

$$q_m(h) = \mathbb{P}(V_m = h \mid a^m_{1:T_m}, s^m_{1:T_m}).$$

In code, the result is:

```
dict[(session_id, hand_id, player_id), dict[hand_class, probability]]
```

12.3 Soft Counts

`expected_action_counts(records, responsibilities)` computes:

$$N_{h,s,a} = \sum_m \sum_t q_m(h) \mathbf{1}\{s_t = s,\ a_t = a\}.$$

This is the central EM bridge: true hand labels are replaced by posterior responsibilities.

12.4 M-Step Helper

`m_step_likelihoods(counts, prior_model, kappa, smoothing)` computes:

$$\phi_{h,s,a} = \frac{N_{h,s,a} + \kappa p_{\mathrm{GTO}}(a \mid h, s) + \epsilon}{\sum_{a'} \left[N_{h,s,a'} + \kappa p_{\mathrm{GTO}}(a' \mid h, s) + \epsilon \right]}.$$

`kappa` controls how strongly the GTO-inspired prior affects the learned likelihood. `smoothing` is additional nonzero mass for numerical stability.

12.5 Evaluation Helper

`true_class_log_loss(records, responsibilities)` computes average:

$$-\log q_m(h^\star_m),$$

where $h^\star_m$ is the true hand class from Pluribus. This metric is only available because Pluribus reveals hole cards.

12.6 What Is Still Missing for Full EM

To run full EM, we still need:

- an iterative training loop;
- train/validation splits;
- model serialization;
- a learned likelihood class that can replace `PreflopGTOPrior` inside the filter after each M-step;
- convergence monitoring based on held-out action log-loss or true-class log-loss.

13 `README.md`

13.1 Role

`README.md` is a short usage guide. It lists the package modules and shows two examples:

1. extracting records and filtering one player-hand;
2. running the EM starter functions.

13.2 Minimal Filtering Example

```
from utils_jin.records import extract_session_records, group_records_by_player_hand
from utils_jin.preflop_filter import filter_preflop_records

records = extract_session_records("pluribus/78", include_postflop=False)
groups = group_records_by_player_hand(records)

key, one_player_hand = next(iter(groups.items()))
filt = filter_preflop_records(one_player_hand)

print(key)
print(filt.top_k(10))
print(filt.steps[-1])
```

13.3 Minimal EM Starter Example

```
from utils_jin.em_preflop import (
    e_step_preflop,
    expected_action_counts,
    m_step_likelihoods,
)

q = e_step_preflop(records)
counts = expected_action_counts(records, q)
phi = m_step_likelihoods(counts)
```

14 Current Smoke-Test Results

The implementation was smoke-tested on the Pluribus data. Full pre-flop extraction produced:

Quantity	**Value**
Pre-flop decision records	61,700
Distinct compact pre-flop states	76
Known hand-class labels	61,700
Exact combos generated	1,326
Pre-flop classes generated	169

The action bucket distribution after re-bucketing was:

Bucket	Count
0: fold	44,077
1: check/call	6,777
2: small bet/raise	8,293
3: medium bet/raise	966
4: large bet/raise	1,587

This confirms that all five model action buckets are currently represented.

15 How the Scripts Fit Together

15.1 Filtering Workflow

```
from utils_jin.records import extract_session_records, group_records_by_player_hand
from utils_jin.preflop_filter import filter_preflop_records

records = extract_session_records("pluribus/78", include_postflop=False)
groups = group_records_by_player_hand(records)

for key, group in groups.items():
    filt = filter_preflop_records(group)
    posterior = filt.range
```

This workflow uses:

- `compat.py` indirectly through `records.py`;
- `records.py` for extraction;
- `action_encoder.py` and `state_encoder.py` inside extraction;
- `cards.py` for true class labels and initial prior;
- `gto_prior.py` for action likelihoods;
- `preflop_filter.py` for posterior updates.

15.2 EM Starter Workflow

```
from utils_jin.em_preflop import (
    e_step_preflop,
    expected_action_counts,
    m_step_likelihoods,
    true_class_log_loss,
)

q = e_step_preflop(records)
counts = expected_action_counts(records, q)
phi = m_step_likelihoods(counts)
loss = true_class_log_loss(records, q)
```

This workflow computes one EM-style E-step and one M-step table. It does not yet loop until convergence.

16 Recommended Next Engineering Steps

1. Add a train/test split utility for Pluribus sessions.
2. Add a likelihood model class that stores learned $\phi_{h,s,a}$ tables and implements the same action-probability interface as `PreflopGTOPrior`.
3. Implement an actual EM loop:

$$\text{E-step} \to \text{soft counts} \to \text{M-step} \to \text{evaluate} \to \text{repeat}.$$

4. Add held-out action log-loss and true-class log-loss metrics.
5. Implement the post-flop range-to-strength mapper:

$$R(h) + \text{board} \to Q(w).$$

6. Add a post-flop strength filter using $\psi_{w,s,a}$.
7. Add learned θ only after the baseline EM loop works.

17 Summary

`utils_jin/` now contains the complete utility layer needed to begin pre-flop filtering and to start implementing EM. The most important design achievement is separation of concerns:

- parser compatibility is isolated in `compat.py`;
- raw poker data is flattened in `records.py`;
- model state/action spaces are controlled by encoders;
- GTO-inspired priors are isolated behind a replaceable interface;
- filtering is independent of extraction;
- EM utilities are built on top of filtering outputs.

This structure lets the project improve one layer at a time without rewriting the entire pipeline.